Data Manipulation(continued)

Using the instance of the following table:
Staff (staffNo, fName, lName, city, position, salary, branchNo)

staffNO	fName	lName	city	position	salary	branchNo
s1	Gad	Ahmed	Tanta	Manager	60,000	B02
s2	Samy	Tamer	Cairo	Assistant	30,000	B02
s3	Samy	Ahmed	Tanta	Supervisor	50,000	B01
s4	Tamer	Samir	Giza	Manager	40,000	B03

Row selection (WHERE clause)

- The previous section shows the use of the SELECT statement to retrieve all rows from a table.
- However, we often need to restrict the rows that are retrieved.
  This can be achieved with the **WHERE** clause, which consists of the keyword **WHERE** followed by a search condition that specifies the rows to be retrieved.

The 5 basic search conditions

(or *predicates* using the ISO terminology) are as follows:

1. ***Comparison****:* Compare the value of one expression to the value of another expression.
2. ***Range****:* Test whether the value of an expression falls within a specified range of values.
3. ***Set membership****:* Test whether the value of an expression equals one of a set of values.
4. ***Pattern match:*** Test whether a string matches a specified pattern.
5. ***Null:*** Test whether a column has a null (unknown) value.

We now present examples of each of these types of search conditions.

Ex 1: Comparison search condition

List all staff with a salary greater than £10,000.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > 30000;
```

- The table is **Staff** and the predicate is **salary > 30000**.
- The selection creates a new table containing only those Staff rows with a salary greater than £30,000.
- The result of this operation is shown in Table 1.

Table 1 Result table for Example 1.

staffNO	fName	lName	position	salary
s1	Gad	Ahmed	Manager	60,000
s3	Samy	Ahmed	Supervisor	50,000
s4	Tamer	Samir	Manager	40,000

In SQL, the following simple comparison operators are available:

= equals
< > is not equal to ! = is not equal to (allowed in some dialects)
< is less than < = is less than or equal to
> is greater than > = is greater than or equal to

- More complex predicates can be generated using the logical operators **AND**, **OR**, and **NOT**, with parentheses (if needed or desired) to show the order of evaluation.
- The rules for evaluating a conditional expression are:
  - an expression is evaluated left to right;
  - sub-expressions in brackets are evaluated first;
  - NOTs are evaluated before ANDs and ORs;
  - ANDs are evaluated before ORs.

The use of parentheses is always recommended to remove any possible ambiguities.

Ex 2: Compound comparison search condition

List the staff residing in Cairo or Giza.

SELECT *
FROM Staff
WHERE city = 'Cairo' **OR** city = 'Giza';

In this example the logical operator OR is used in the WHERE clause to find the *staff* in Cairo (city = 'Cairo') *or* in Giza (city = ' Giza').

- The result table is shown in Table 2.

Table 2 Result table for Example 2.

staffNO	fName	lName	City	position	salary	branchNo
s2	Samy	Tamer	Cairo	Assistant	30,000	B02
s4	Tamer	Samir	Giza	Manager	40,000	B03

Ex 3: Range search condition (BETWEEN / NOT BETWEEN)

List all staff with a salary between £20,000 and £30,000.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary BETWEEN 30000 AND 40000;
```

- The BETWEEN test includes the endpoints of the range, so any members of staff with a salary of £30,000 or £40,000 would be included in the result.
- The result table is shown in Table 3.

Table 3 Result table for Example 3.

staffNO	fName	lName	position	salary
s2	Samy	Tamer	Assistant	30,000
s3	Samy	Ahmed	Supervisor	50,000
s4	Tamer	Samir	Manager	40,000

We could have expressed the above query as:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 30000 AND salary <= 40000;
```

Ex 4: Set membership search condition (IN/NOT IN)

List of all managers and supervisors.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');
```

- The set membership test (IN) tests whether a data value matches one of a list of values.
- In this case either 'Manager' or 'Supervisor'.
- The result table is shown in Table 4.

Table 4 Result table for Example 4.

staffNO	fName	lName	position
s1	Gad	Ahmed	Manager
s3	Samy	Ahmed	Supervisor
s4	Tamer	Samir	Manager

We could have expressed the above query as:

```
SELECT staffNo, fName, lName, position.
FROM Staff
WHERE position = 'Manager' OR position = 'Supervisor';
```

However, the IN test provides a more efficient way of expressing the search condition, particularly if the set contains many values.